

Activity: Train ViT for Classification

Introduction

In this activity, you will fine-tune a **Vision Transformer (ViT)** (`google/vit-base-patch16-224`) and a **Convolutional Neural Network (CNN)** (ResNet-50) on the **CIFAR-10 dataset** for image classification, comparing their performance in terms of accuracy and training time. This exercise builds on concepts from lessons on CNNs, data augmentation, feature extraction, and Vision Transformers, where ViTs were introduced as leveraging global attention mechanisms to process images as sequences of patches, unlike CNNs' localized feature extraction. You will use pretrained models from Hugging Face and torchvision, apply data augmentation, and train for 2 epochs with a batch size of 32 to fit within a 2-hour timeframe. The results will be visualized in a table and bar chart to analyze trade-offs between accuracy and computational efficiency, preparing you for selecting appropriate models for real-world computer vision tasks.

Learning Outcomes

By the end of this activity, you will:

1. Load and preprocess the CIFAR-10 dataset for ViT and CNN training.
2. Fine-tune a pretrained Vision Transformer and ResNet-50 for CIFAR-10 classification.
3. Train both models and evaluate their performance (test accuracy and training time).
4. Visualize and compare results using a table and bar chart.
5. Analyze how ViT's global attention mechanism and CNN's convolutional approach impact performance and efficiency.

How to Run Your `.ipynb` File on Colab (Recommended)

Option 1: Upload Directly

1. Go to <https://colab.research.google.com>
2. Click **"Upload"** tab → Select your `.ipynb` file from your computer
3. Colab will open it in a new tab
4. Make sure to go to `Runtime` → `Change runtime type` and set:
 - **Hardware accelerator:** `GPU`

Option 2: Mount from Google Drive

If your notebook is stored in Drive:

1. Open Colab
 2. Choose **"Google Drive"** tab
 3. Navigate to the `.ipynb` file and open it
-

Before Running Any Cell

Add a **setup cell at the top** of your notebook to ensure dependencies are installed:

```
!pip install -q torch torchvision transformers datasets matplotlib numpy timm datasets
```

Then Run Cell-by-Cell

1. Start with installation
2. Set up device check and conditional DataLoader
3. Proceed through data, model, training, and evaluation

Optional: Save Outputs Back to Google Drive

If you want to save trained model or results:

```
from google.colab import drive
drive.mount('/content/drive')

# Example to save
torch.save(model_vit.state_dict(), '/content/drive/MyDrive/vit_cifar10.pth')
```

Otherwise if you are using your machine follow the following Steps:

Working with Dev Container

To complete this assignment in a reliable and fully configured environment, please refer to the instructions in the file: `README-devcontainer.md`. This guide walks you through opening the assignment in **Visual Studio Code** using a **Dev Container**, which automatically installs all necessary Python libraries and tools. Following that setup ensures that the notebook runs smoothly without manual configuration or missing dependencies. Make sure to open the **main activity folder** in VS Code and follow the steps outlined in the Dev Container guide before starting the notebook.

Starter File

- Open the notebook: `activity_vit_for_classification.ipynb` in the `code/start` folder.
- Run it locally or in Google Colab with GPU support (preferred for faster training).
- Required libraries: `torch`, `torchvision`, `transformers`, `matplotlib`, `numpy`.
- Use the CIFAR-10 dataset, automatically loaded via `torchvision`.

Activity Code Steps

This is a guided learning activity. You'll:

1. **Install Dependencies**

2. Install necessary libraries (`torch`, `torchvision`, `transformers`, `matplotlib`, `numpy`) to support ViT and ResNet-50 training. Ensure GPU availability is checked for faster computation.

3. Load and Preprocess the CIFAR-10 Dataset

Load the CIFAR-10 dataset (50,000 training and 10,000 test images, 32x32 RGB) using `torchvision`. Apply specific preprocessing for each model:

- **ViT:** Resize images to 224x224 (required by `google/vit-base-patch16-224`), normalize using `ViTImageProcessor`, and apply random horizontal flips for augmentation.
- **ResNet-50:** Keep images at 32x32, normalize with mean and standard deviation of 0.5, and apply random horizontal flips and random crops (padding=4) for augmentation.
- Create `DataLoader`s with a batch size of 32, `num_workers=2`, and `pin_memory=True` for efficient data loading.

4. Define Training and Evaluation Functions

Implement functions to train and evaluate both models:

- **Training:** Use Adam optimizer (learning rate 1e-4) and cross-entropy loss, train for 2 epochs, and track training time. Handle ViT's `ImageClassifierOutput` (accessing `logits`) and ResNet-50's raw tensor output.
- **Evaluation:** Compute test accuracy by comparing predicted labels to true labels.
- Clear GPU memory before training to prevent crashes.

5. Fine-Tune Vision Transformer

Load the pretrained `google/vit-base-patch16-224` model, adjust the classifier for 10 CIFAR-10 classes using `ignore_mismatched_sizes=True` to handle ImageNet's 1000-class mismatch. Fine-tune the model on CIFAR-10, train for 2 epochs, and evaluate test accuracy. Expect ~90–97% accuracy and longer training time (~2000–3500 seconds on GPU) due to ViT's global attention mechanism (~8B FLOPs).

6. Fine-Tune ResNet-50

Load the pretrained ResNet-50 model, replace the final fully connected layer for 10 classes, and fine-tune on CIFAR-10 for 2 epochs. Expect ~88–90% accuracy and faster training time (~1000–1500 seconds) due to its efficient architecture (~4B FLOPs) and smaller 32x32 images.

7. Compare and Visualize Results

Compare ViT and ResNet-50 based on test accuracy and training time. Display results in a table and create a bar chart to visualize test accuracy for both models. Example table format:

Model	Accuracy (%)	Training Time (s)
ViT-B/16	96.41	3222.61
ResNet-50	80.83	148.54

8. Analyze Results

- Evaluate test accuracy and training time for both models.
- Analyze why ViT may outperform ResNet-50 (e.g., global attention vs. localized convolutions, larger input size).
- Assess the trade-off between accuracy and computational cost (training time).

- Consider how model architecture and input size affect performance, referencing lessons on ViTs and CNNs.

Troubleshooting Tips

- **RAM Crash:** If you encounter memory issues with ViT, reduce the batch size (e.g., to 16) or use fewer epochs.
- **Low Accuracy:** Increase training epochs or adjust the learning rate (e.g., try 5e-5 or 2e-4) to improve performance.
- **Errors:** Ensure all libraries (`torch`, `torchvision`, `transformers`, `matplotlib`, `numpy`) are installed and imported correctly.
- **Slow Training:** Use a GPU (e.g., Google Colab's T4) to speed up training, especially for ViT.

Conclusion

In this activity, you:

- Loaded and preprocessed the CIFAR-10 dataset for ViT and ResNet-50 with appropriate transforms.
- Fine-tuned a pretrained Vision Transformer and ResNet-50 for CIFAR-10 classification.
- Trained both models for 2 epochs and evaluated their test accuracy and training time.
- Visualized results in a table and bar chart to compare performance.
- Analyzed the impact of ViT's global attention mechanism and ResNet-50's convolutional approach on accuracy and efficiency.

You've gained insights into the strengths and weaknesses of ViTs and CNNs. ViT achieved higher accuracy (e.g., 96.41%) due to its global attention and larger 224x224 input size, but required significantly longer training time (e.g., 3222.61 seconds) compared to ResNet-50 (e.g., 80.83% accuracy, 148.54 seconds). This aligns with the lesson on Vision Transformers, which emphasized their computational intensity but superior performance on complex datasets, and CNN coursework, which highlighted efficiency for smaller images. These findings enhance your ability to choose appropriate models for computer vision tasks, balancing accuracy and computational resources, and prepare you for advanced applications like object detection or attention map visualization.